



ЛЕКЦИЯ 08

Решающие деревья, случайные леса и градиентный бустинг

Курс «Машинное обучение и безопасность систем»

Магистратура НИУ ВШЭ (МИЭМ) · ОП «Информационная безопасность и технологии ИИ»

Автор: Силаев Ю. В. *Время изучения: ~75 минут · 59 слайдов*



О чём L08 и зачем она в курсе

Деревья и ансамбли – это первый класс **нелинейных моделей для табличных данных** в нашем курсе. До сих пор у вас был линейный baseline (L06) и инструменты его контроля – регуляризация и порог (L07). Линейная модель проводит одну разделяющую границу; но реальные табличные признаки в задачах ИБ – частоты, объёмы, тайминги соединений – связаны нелинейно и взаимодействуют друг с другом. Дерево разбивает пространство признаков на области правилами «если-то» и потому ловит такие зависимости естественным образом.

КЛЮЧЕВОЙ ТЕЗИС

Деревья и ансамбли могут существенно улучшать качество на табличных данных, но требуют строгого контроля переобучения, честного сравнения с baseline и осторожной интерпретации важности признаков.



Учебные результаты

Вы научитесь:

- объяснять, как дерево разбивает пространство признаков на области;
- записывать критерии разбиения Gini и entropy формулами и понимать, что они измеряют;
- объяснять, почему одиночное дерево склонно к переобучению;
- описывать идею bagging и случайного леса;
- описывать boosting как последовательное исправление ошибок;
- различать Random Forest и Gradient Boosting на прикладном уровне;
- называть ключевые гиперпараметры деревьев и ансамблей;
- читать feature importance с учётом её ограничений (детально – в L09);
- сравнивать ансамбли с линейным baseline по честному протоколу.



Что нужно знать до начала

L08 опирается на четыре предыдущие лекции

Эта лекция предполагает, что вы уже владеете материалом блока 1.

L04

Метрики качества. accuracy, precision/recall, ROC-AUC и цена ошибок – чем мерить качество модели.

L05

Валидация и утечки. split, кросс-валидация и понятие утечки данных (data leakage). В L08 термин используется без переопределения – как в L05.

L06

Линейные модели. линейная модель и log-loss – наш baseline, с которым обязательно сравнивается любое дерево или ансамбль.

L07

Регуляризация и порог. понятие переобучения и контроля сложности – центральная идея, которую дерево доводит до предела.



План лекции

5 / 59

Часть 1 · Введение

7 частей · 59 слайдов · ~75 минут самостоятельного изучения

1	Введение	слайды 1-5	~6 мин
2	Мотивация: зачем нужны деревья	слайды 6-9	~5 мин
3	Основное содержание: деревья, леса, бустинг, importance	слайды 10-46	~48 мин
4	Связки с практикой ИБ (CIC-IDS-2018)	слайды 47-53	~9 мин
5	Итоги	слайды 54-56	~3 мин
6	Источники для углубления	слайды 57-58	~2 мин
7	Глоссарий	слайды 59	~2 мин



Почему линейного baseline мало

6 / 59

Часть 2 · Мотивация

Табличные ИБ-признаки нелинейны и взаимодействуют

Линейная модель из L06 принимает решение по взвешенной сумме признаков $\langle \mathbf{w}, \mathbf{x} \rangle + \mathbf{b}$: каждый признак вносит вклад независимо и строго пропорционально своему значению. Это сильное предположение. В реальных табличных данных ИБ оно почти всегда нарушается, и именно поэтому линейка часто упирается в потолок качества.

НЕЛИНЕЙНОСТЬ

Риск растёт не пропорционально значению признака. 500 соединений в минуту – не «в пять раз подозрительнее», чем 100: до порога это норма, за порогом – почти наверняка скан портов. Линейный вес такую ступеньку не выразит.

ВЗАИМОДЕЙСТВИЯ

Опасность определяется сочетанием признаков. Ночное время + внешний IP + редкий порт подозрительно вместе, а каждый по отдельности – нет. Линейка складывает вклады, но не умеет сказать «опасно, только если всё сразу».

Дерево решает обе проблемы по построению: разбиения по порогам дают ступеньки (нелинейность), а вложенные условия «если А и В» – взаимодействия. Отсюда и интерес к деревьям на табличных данных.



ИБ-кейс: приоритизация алертов

7 / 59

Часть 2 · Мотивация

Очередь SOC – где линейка недотягивает и почему

ИБ-КЕЙС

Ситуация. SOC получает тысячи алертов в сутки. Аналитиков хватает на проверку ~150. Нужна модель, которая упорядочит очередь: реальные инциденты – наверх, шум – вниз. Признаки агрегатные и табличные: число сработавших правил, тип источника, время суток, частота для этого хоста за неделю, репутация IP, объём переданных данных.

Линейная модель (L06) на этих признаках. $\text{precision@150} \approx 0.34$ – лишь треть проверенных алертов оказались реальными. Логистическая регрессия не ловит, что «много правил» опасно только в сочетании с внешним источником в нерабочее время.

Запрос инженера: модель, которая поднимет precision@150 , потому что каждое место в очереди из 150 – это рабочее время аналитика (линия «цена ошибок и порог», L04 → L07). Именно здесь естественно появляются деревья и ансамбли – к ним и переходим.



Три риска, которые приходят вместе с мощностью модели

Соблазн прост: раз линейка упёрлась – возьмём модель помощнее, обучим, посмотрим на метрику. Но рост числа параметров приносит не только качество. Мощная модель так же легко выучивает **шум и артефакты данных**, как и полезный сигнал. Прежде чем разбирать деревья, зафиксируем три риска, которые будут красной нитью всей лекции.

1 Переобучение

Глубокая модель подгоняется под обучающую выборку, включая её случайности. На train – почти идеал, на новых данных – провал. Контролируется глубиной, размером листа, регуляризацией (L07).

2 Утечка данных

Признак, незаметно содержащий ответ, даёт фантастическую метрику, которая исчезает в проде. Чем мощнее модель, тем охотнее она цепляется за утечку (как в L05).

3 Ложная уверенность

Высокая важность признака читается как «доказали причину». Это не так: importance показывает, на что опиралась модель, а не что влияет на мир (детальный разбор – в L09).



Частая ошибка: «получили 0.99 – значит, готово»

Антипаттерн, с которым борется вся эта лекция

⚠ ЧАСТАЯ ОШИБКА

«Запустили градиентный бустинг „из коробки“, получили ассурасу 0.99 на отложенной выборке – модель отличная, задача решена». Так рассуждают чаще всего, и почти всегда это иллюзия: высокая метрика без проверки протокола ничего не доказывает.

ПОЧЕМУ ЭТО НЕВЕРНО

0.99 на самом деле может означать утечку, а не качество. Если split сделан случайно по строкам, а в данных есть дубликаты потоков или временная зависимость (L05), модель «подсматривает» в почти такие же примеры. Если в признаках есть прокси таргета – метрика взлетает по утечке. А при сильном дисбалансе классов сама ассурасу обманчива: предсказывая всегда «Benign», легко получить 0.99 (L04).

Дальше → разбираем деревья и ансамбли так, чтобы каждый раз отделять настоящее качество от иллюзии: контроль переобучения, честный split, сравнение с baseline.



Б Л О К 3 А

Решающее дерево: как оно устроено

Разберём, как дерево разбивает пространство признаков правилами «если-то», что такое узел, лист и глубина, и как по этой структуре получается предсказание. Это фундамент, на котором дальше строятся леса и бустинг.

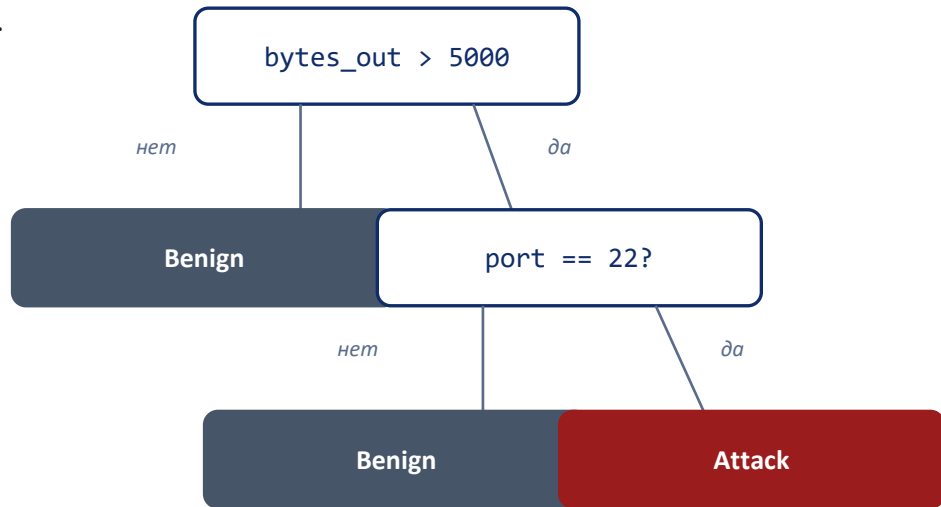


Дерево как последовательность вопросов

Узел, ветвь, лист, глубина

Решающее дерево – это набор вложенных правил «если-то». Каждый **внутренний узел** задаёт условие по одному признаку (например, `bytes_out > 5000`). Ответ «да/нет» отправляет объект в одну из двух **ветвей**. Так объект спускается от корня вниз, пока не попадёт в **лист** – конечный узел без условий, где хранится ответ модели.

Глубина – длина самого длинного пути от корня к листу. Чем глубже дерево, тем более тонкие комбинации условий оно может выразить – и тем легче переобучается (слайд 18).



Корень → внутренние узлы → листья. Глубина этого дерева = 3.



Как дерево выдаёт предсказание

Путь одного объекта от корня к листу

Возьмём один сетевой поток с признаками `bytes_out = 8200`, `port = 22` и проследим его путь по дереву со слайда 11. Предсказание – это просто ответ того листа, в который объект в итоге попал.

1**Корень**`bytes_out > 5000 ?``8200 > 5000 → да, идём вправо`**2****Узел 2**`port == 22 ?``22 == 22 → да, идём вправо`**3****Лист**

– (конечный узел)

ответ листа: Attack

Для классификации лист хранит класс (или доли классов → вероятность). **Для регрессии** – среднее значение целевой переменной по обучающим объектам этого листа.



Что дерево оптимизирует при разбиении

Impurity – мера «загрязнённости» узла

ОПРЕДЕЛЕНИЕ

Impurity (загрязнённость узла)

Числовая мера того, насколько перемешаны классы среди объектов узла. Узел «чистый», если в нём объекты только одного класса ($\text{impurity} = 0$), и максимально «грязный», если классы представлены поровну.

Зачем это нужно. Дерево строит разбиения жадно: на каждом шаге выбирает условие, которое сильнее всего снижает impurity дочерних узлов по сравнению с родителем. Иными словами – ищет вопрос, который лучше всего разделяет классы. Осталось задать impurity формулой: это и делают два критерия – Gini и энтропия (следующие два слайда).



чистый $G=0.0$



смесь 70/30 $G=0.42$



50/50 $G=0.5$



Первый из двух способов измерить загрязнённость

$$G = 1 - \sum_k p_k^2$$

p_k – доля объектов класса k в узле; сумма берётся по всем классам. $\sum_k p_k = 1$. Gini – это вероятность того, что два случайно взятых из узла объекта окажутся разных классов.

ЧИСТЫЙ УЗЕЛ (один класс)

$$p = (1, 0): \quad G = 1 - (1^2 + 0^2) = 0$$

Минимум. Делить дальше незачем – узел готов стать листом.

РАВНОМЕРНАЯ СМЕСЬ 50/50

$$p = (\frac{1}{2}, \frac{1}{2}): \quad G = 1 - (\frac{1}{4} + \frac{1}{4}) = 0.5$$

Максимум для двух классов. Узел максимально неопределён.



Второй критерий и то, как дерево выбирает разбиение

$$H = - \sum_k p_k \cdot \log_2 p_k$$

Энтропия измеряет неопределённость в битах. На чистом узле $H = 0$; на смеси 50/50 двух классов $H = 1$ бит (максимум). Поведение качественно такое же, как у Gini.

$$IG = H(\text{родитель}) - \sum (N_{\text{дочь}} / N) \cdot H(\text{дочь})$$

Information gain – это уменьшение impurity после разбиения: насколько «чище» стали дочерние узлы по сравнению с родителем, взвешенно по их размеру. **Дерево на каждом шаге выбирает условие с наибольшим information gain** – то есть вопрос, который сильнее всего разделяет классы. Тот же принцип работает и для Gini.



Gini или энтропия – что выбрать?

16 / 59

Часть 3 · Основное содержание

Короткий ответ: на практике почти всё равно

Оба критерия устроены одинаково по сути: **минимальны на чистом узле и максимальны на равномерной смеси классов**. Они монотонно связаны и в большинстве задач выбирают одни и те же или очень близкие разбиения. Разница в итоговом качестве дерева, как правило, теряется на фоне куда более важных решений – глубины, размера листа, честности валидации.

Gini

Чуть быстрее: не нужно вычислять логарифм. Значение по умолчанию в большинстве реализаций деревьев.

Энтропия

Опирается на теоретико-информационную трактовку (биты неопределённости). Результат практически тот же, вычислений чуть больше.

Вывод для практики: не тратьте время на выбор между Gini и entropy. Берите значение по умолчанию и направьте усилия на контроль переобучения – об этом следующий блок.



Б Л О К 3 В

Почему дерево переобучается и как это контролировать

Одинокое дерево, если его не ограничить, выучивает обучающую выборку наизусть – вместе с её шумом. Разберём механизм этого переобучения и четыре гиперпараметра, которыми инженер удерживает сложность под контролем. Это прямое продолжение линии регуляризации из L07.



Почему дерево переобучается

18 / 59

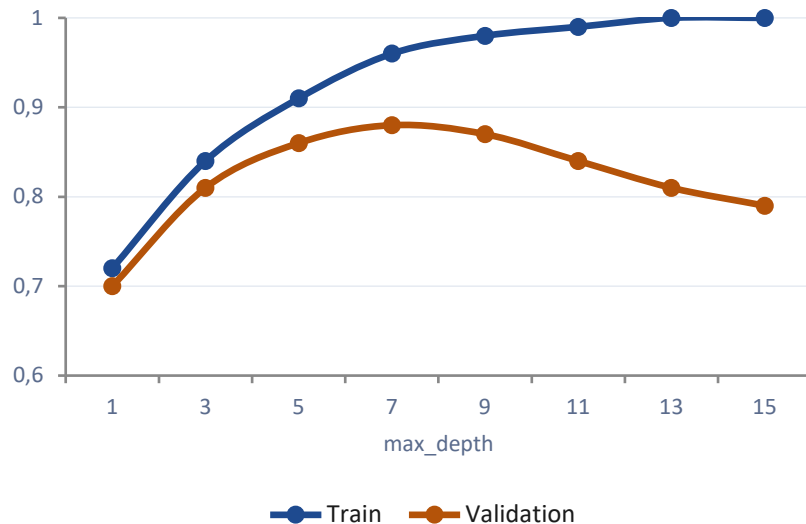
Часть 3 · Основное содержание

Чем глубже – тем больше подгонки под шум

Если дереву разрешить расти без ограничений, оно будет дробить узлы до тех пор, пока в каждом листе не останутся объекты одного класса. На обучающей выборке это даёт **идеальную точность** – но ценой того, что последние разбиения отделяют уже не классы, а **случайный шум** конкретных примеров.

Такое дерево описывает не закономерность, а саму таблицу. На новых данных эти «выученные наизусть» правила не работают – **validation-качество падает**, хотя train-качество почти идеально. Это и есть переобучение (понятие из L07).

Точность по глубине дерева



Train растёт к 1.0, validation разворачивается вниз – зазор между ними и есть переобучение.



Пример: дерево «наизусть»

Что показывают цифры при росте глубины

Учим дерево на 5000 размеченных потоков, проверяем на отложенных 2000 (честный split, L05). Меняем только **max_depth** – глубину. Смотрим accuracy на train и на validation.

depth = 3	train 0.86	val 0.85	зазор 0.01 – недообучение, но честно
------------------	------------	----------	--------------------------------------

depth = 7	train 0.96	val 0.88	зазор 0.08 – разумный баланс
------------------	------------	----------	------------------------------

depth = 15	train 1.00	val 0.79	зазор 0.21 – явное переобучение
-------------------	------------	----------	---------------------------------

Читаем правильно: растущий зазор train – val – главный сигнал переобучения. Глубокое дерево с train 1.00 не «лучшая модель», а наименее надёжная из трёх.



Чем удержат сложность дерева

Четыре ключевых гиперпараметра контроля

Все они ограничивают, насколько глубоко и мелко дерево может дробить данные. По сути это та же регуляризация, что в L07, только выраженная структурой дерева.

max_depth

Максимальная глубина дерева. Прямой потолок сложности: ограничивает длину цепочек условий и число листьев.

min_samples_leaf

Минимум объектов в листе. Запрещает листья на одном-двух примерах – именно они ловят шум.

min_samples_split

Минимум объектов, чтобы вообще делить узел. Останавливает дробление маленьких узлов.

max_features

Сколько признаков рассматривать при поиске разбиения. Снижает переобучение и пригодится в лесах (блок 3С).



Pruning: обрезка переусложнённого дерева

Альтернативный путь контроля – постобработка

Гиперпараметры со слайда 20 ограничивают рост дерева **заранее** (pre-pruning). Есть и второй подход – **post-pruning**: сначала вырастить дерево целиком, а затем срезать ветви, которые почти не улучшают качество на отложенных данных, но добавляют сложность.

PRE-PRUNING (предобрезка)

Останавливаем рост сразу: задаём `max_depth`, `min_samples_leaf` и т. д. Быстро и дёшево, но можно «передавить» – не дать дереву найти полезное глубокое разбиение.

POST-PRUNING (постобрезка)

Растим полностью, потом срезаем лишнее (например, `cost-complexity pruning` с параметром `ccp_alpha`). Аккуратнее, но дороже по вычислениям. Идею достаточно понимать концептуально.

На практике чаще начинают с *pre-pruning* (быстро, понятно), а *post-pruning* применяют, когда нужно выжать максимум из одиночного дерева. В ансамблях (далее) острота проблемы снижается иначе – усреднением.



Частая ошибка: «на train 100% – модель готова»

Самое распространённое заблуждение про деревья

ЧАСТАЯ ОШИБКА

«Дерево показало ассурасу 1.00 на обучающей выборке – отличная модель». train-метрику принимают за качество модели и не смотрят на validation вообще.

ПОЧЕМУ ЭТО НЕВЕРНО

train-качество почти ничего не говорит о пригодности модели. Любое неограниченное дерево достигает 1.00 на train – просто запомнив выборку. Качество модели определяется только на данных, которых она не видела: на validation и test. Цифра 1.00 на train – это не достижение, а тревожный признак: скорее всего, дерево переобучено и его нужно ограничить (слайд 20) или подрезать (слайд 21).



Мини-резюме: дерево и контроль

Что разобрали в блоках 3А и 3В

Дерево разбивает пространство признаков вложенными правилами «если-то»; объект спускается от корня к листу, лист хранит ответ.

Качество разбиения измеряют impurity: Gini ($G = 1 - \sum p_k^2$) и энтропия ($H = -\sum p_k \log_2 p_k$). На практике выбор между ними почти не важен.

Дерево выбирает разбиение по наибольшему information gain – наибольшему снижению impurity.

Неограниченное дерево переобучается: train $\rightarrow$ 1.00, validation падает. Растущий зазор train – val – главный сигнал.

Сложность контролируют гиперпараметрами (max_depth, min_samples_leaf, min_samples_split, max_features) и pruning.

train-метрика не доказывает качество. Решает только validation / test.



БЛОК 3С

Bagging и случайный лес

Одиночное дерево хрупко: чуть изменишь данные – изменится всё дерево. Идея ансамбля проста: обучить много деревьев на слегка разных выборках и усреднить их ответы. Так появляется Random Forest – рабочая лошадь для табличных задач. Разберём, почему усреднение снижает переобучение и зачем лесу случайность по признакам.



Bagging: усреднение снижает разброс

Часть 3 · Основное содержание

Bootstrap-выборки и голосование деревьев

Bagging (bootstrap aggregating) строит много деревьев, каждое – на своей **bootstrap-выборке**: из обучающих N объектов случайно берут N штук с возвращением, так что часть примеров повторяется, а часть не попадает вовсе.

Ответы деревьев **усредняют** (голосование для классификации, среднее для регрессии). Отдельные деревья ошибаются по-разному, и при усреднении их случайные ошибки гасят друг друга – **разброс модели падает**, а вместе с ним и переобучение.





Bagging деревьев плюс случайность по признакам

ОПРЕДЕЛЕНИЕ

Random Forest (случайный лес)

Ансамбль деревьев, обученных по bagging, с дополнительным приёмом: в каждом узле кандидаты для разбиения выбираются не из всех признаков, а из случайного подмножества (`max_features`). Итоговый ответ – усреднение по всем деревьям.

Два источника случайности. Лес перемешивает данные дважды: разные bootstrap-выборки (как в обычном bagging) и разные подмножества признаков в каждом узле. Именно вторая случайность отличает Random Forest от простого bagging деревьев – и, как увидим на следующем слайде, она делает деревья по-настоящему разными.



Декорреляция деревьев усиливает ансамбль

Усреднение снижает разброс тем сильнее, чем меньше деревья **коррелируют** между собой. Если бы все деревья видели все признаки, они бы раз за разом выбирали один и тот же сильный признак в корень — и получились бы почти одинаковыми. Усреднять копии бессмысленно: разброс так не уменьшить.

ВСЕ ПРИЗНАКИ ДОСТУПНЫ ВСЕМ

Деревья почти одинаковы: один доминирующий признак всегда уходит в корень. Ансамбль $\approx$ одно дерево, выигрыш от усреднения почти нулевой.

СЛУЧАЙНОЕ ПОДМНОЖЕСТВО ПРИЗНАКОВ

В разных узлах доступны разные признаки, поэтому деревья развиваются по-разному и ошибаются независимо. Их ошибки гасятся при усреднении — ансамбль ощутимо лучше любого дерева.

Цена декорреляции: *каждое отдельное дерево чуть слабее (видит меньше признаков), но ансамбль в целом — заметно сильнее и устойчивее.*



«Бесплатная» проверка внутри леса

При построении каждого дерева **около трети объектов** не попадают в его bootstrap-выборку – они «out-of-bag» (OOB) для этого дерева. Идея OOB-оценки: для каждого объекта собрать предсказания только тех деревьев, которые его не видели, и сравнить с истинной меткой.

ОПРЕДЕЛЕНИЕ

ООБ-оценка — оценка качества леса по объектам, не входившим в bootstrap соответствующих деревьев. Получается почти бесплатно, прямо в процессе обучения.

⚠ OOB удобна, но не заменяет честный протокол. Если в данных есть временная или групповая структура (L05), OOB по случайным bootstrap-выборкам так же подвержена утечке, как и случайный split. Для ИБ-данных финальную оценку всё равно делаем по правильному split.



Что настраивать и почему лес прощает ошибки

Random Forest известен тем, что хорошо работает «из коробки»: значения по умолчанию обычно дают сильный результат, а качество слабо чувствительно к точной настройке.

n_estimators

Число деревьев. Больше – стабильнее ответ; качество выходит на плато, дальше растёт только время. Уменьшать его не нужно: лес не переобучается от лишних деревьев.

max_features

Размер случайного подмножества признаков в узле. Главный регулятор декорреляции. Типично $\sqrt{d}$ для классификации.

max_depth / min_samples_leaf

Ограничения на отдельные деревья. В лесу можно позволить деревьям быть глубже, чем одиночному: усреднение страхует.

bootstrap

Включает bagging (по умолчанию да). Нужен, чтобы деревья обучались на разных выборках и появлялась OOB-оценка.



Что разобрали в блоке 3С

Bagging обучает много деревьев на bootstrap-выборках и усредняет ответы; усреднение гасит случайные ошибки и снижает разброс.

Random Forest = bagging + случайное подмножество признаков в каждом узле.

Случайность по признакам декоррелирует деревья: каждое чуть слабее, но ансамбль заметно сильнее и устойчивее.

ООВ-оценка даёт почти бесплатную проверку по «невидимым» объектам, но не заменяет честный split на структурированных данных.

Лес хорошо работает «из коробки» и не переобучается от добавления деревьев; главный регулятор – `max_features`.



БЛОК 3D

Градиентный бустинг

Лес строит деревья параллельно и усредняет их. Бустинг идёт другим путём: деревья строятся последовательно, и каждое следующее исправляет ошибки предыдущих. Это часто даёт самое сильное качество на табличных данных – но и требует более аккуратной настройки, иначе модель переобучается. Разберём идею, схему обновления и ключевые гиперпараметры.



Последовательно, а не параллельно

Bagging и boosting – два разных способа собрать ансамбль. **Bagging** строит деревья независимо и параллельно, борясь с разбросом. **Boosting** строит их по цепочке: каждое новое дерево обучается на том, где ансамбль пока ошибается, и постепенно уменьшает систематическую ошибку.

BAGGING / RANDOM FOREST

Деревья: независимы, параллельно.

Цель: снизить разброс усреднением.

Деревья: глубокие, сильные по отдельности.

Лишние деревья: не вредят.

GRADIENT BOOSTING

Деревья: последовательно, по цепочке.

Цель: снизить систематическую ошибку.

Деревья: мелкие, слабые по отдельности.

Лишние деревья: могут переобучить.



Ансамбль, который строится по антиградиенту

ОПРЕДЕЛЕНИЕ

Gradient Boosting (градиентный бустинг)

Метод последовательного построения ансамбля: на каждом шаге добавляется новое небольшое дерево, которое приближает направление наискорейшего убывания функции потерь (антиградиент) – то есть исправляет текущие ошибки модели.

Интуиция без формул. Представьте, что модель – это сумма поправок. Сначала есть грубое приближение. Смотрим, где и насколько оно ошибается, и обучаем маленькое дерево предсказывать именно эту ошибку. Прибавляем его (с небольшим весом) к модели. Повторяем: каждая итерация чуть-чуть подтягивает предсказание к правде. Слово «градиентный» означает, что направление поправки выбирается по градиенту функции потерь – но для практики достаточно образа «каждое дерево чинит остаток ошибки».



Схема обновления и роль *learning rate*

$$F_m(x) = F_{m-1}(x) + v \cdot h_m(x)$$

F_{m-1}

ансамбль после $m-1$ шагов – то, что уже построено.

h_m

новое дерево на шаге m , обученное исправлять ошибки F_{m-1} .

v

learning rate ($0 < v \leq 1$): насколько сильно прибавляем новое дерево.

F_m

обновлённый ансамбль после добавления h_m .

Маленький v = осторожные шаги. Низкий *learning rate* делает каждую поправку слабой, поэтому шагов (деревьев) нужно больше – но итог обычно точнее и устойчивее. Это главная связка гиперпараметров бустинга (следующий слайд).



Связаны между собой – настраивать вместе

В отличие от леса, бустинг **чувствителен к настройке**: здесь лишние деревья и слишком глубокие базовые модели приводят к переобучению. Параметры связаны и настраиваются совместно.

`learning_rate (v)`

Вес каждого дерева. Меньше – точнее и устойчивее, но нужно больше деревьев. Ключевой параметр.

`n_estimators`

Число деревьев = число шагов. Слишком много при большом $v \rightarrow$ переобучение. Связан с `learning_rate` обратно.

`max_depth`

Глубина базовых деревьев. В бустинге держат маленькой (2-6): нужны «слабые» модели, а не сильные.

`subsample`

Доля объектов на каждом шаге ($<1 \rightarrow$ стохастический бустинг). Добавляет случайность, снижает переобучение.



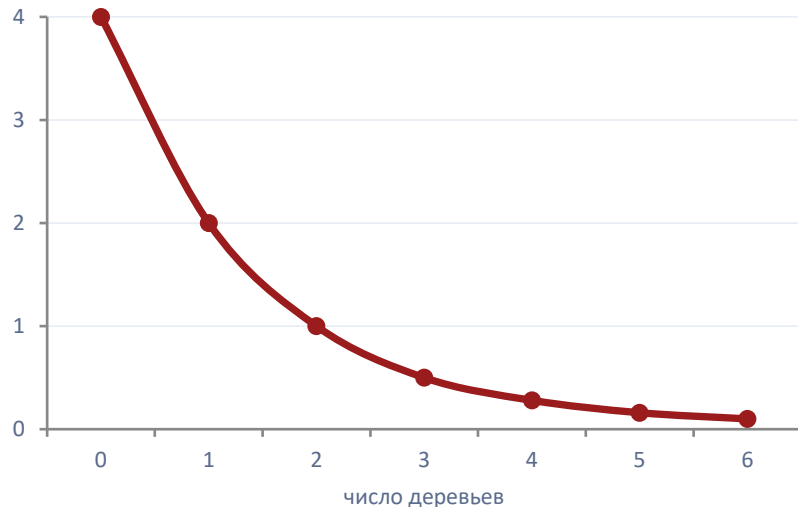
Пример: как убывает ошибка

Несколько шагов бустинга на регрессии

Учим бустинг предсказывать число (для наглядности — регрессия). Истинное значение объекта $y = 10$, learning rate $v = 0.5$. Смотрим, как предсказание подтягивается к 10 шаг за шагом.

старт	→ 6.0	ошибка 4.0
+ дерево 1	→ 8.0	ошибка 2.0
+ дерево 2	→ 9.0	ошибка 1.0
+ дерево 3	→ 9.5	ошибка 0.5

Ошибка по числу деревьев



Каждое дерево срезает часть остатка; ошибка падает, но всё медленнее.



Частая ошибка: «бустинг всегда лучше»

37 / 59

Часть 3 · Основное содержание

Сильнее ≠ всегда правильный выбор

⚠ ЧАСТАЯ ОШИБКА

«Бустинг – самый мощный метод, поэтому всегда берём его вместо случайного леса». Выбор делают по «репутации» метода, а не по результату на своих данных.

ПОЧЕМУ ЭТО НЕВЕРНО

что лучше – зависит от данных, шума и бюджета на настройку. При сильном шуме в метках бустинг легко переобучается, последовательно подгоняясь под ошибки разметки, – лес здесь устойчивее. Бустинг чувствительнее к гиперпараметрам, дольше настраивается и хуже параллелится. Random Forest часто даёт сравнимое качество «из коробки» за минуты. Правильный ход – сравнить оба честно (на одном split, с baseline из L06), а не назначать победителя заранее.



Что разобрали в блоке 3D

Boosting строит деревья последовательно: каждое исправляет ошибки уже построенного ансамбля.

Обновление: $F_m = F_{m-1} + v \cdot h_m$. Новое дерево h_m приближает антиградиент потерь, v – learning rate.

Базовые деревья делают мелкими (2-6); сила – в большом числе слабых поправок, а не в одном сильном дереве.

Главная связка: меньше learning_rate → больше n_estimators; max_depth и subsample сдерживают переобучение.

Бустинг мощный, но чувствителен к настройке и переобучается от лишних деревьев – не «всегда лучше» леса.



БЛОК 3Е

Выбор модели и важность признаков

Два практических вопроса в финале основного содержания. Первый: как выбирать между Random Forest и Gradient Boosting и что делать с пропусками и категориальными признаками. Второй: как читать feature importance – и почему её так легко переоценить. Важность признаков мы вводим только концептуально; систематический разбор интерпретации – в следующей лекции L09.



Практический выбор между двумя ансамблями

Оба – сильные ансамбли деревьев, но с разным характером. Выбор зависит не от «репутации», а от данных и от того, сколько времени есть на настройку.

RANDOM FOREST

- Хорош «из коробки», мало настройки.
- Устойчив к шуму в метках.
- Легко параллелится, быстро обучается.
- Не переобучается от лишних деревьев.
- Качество обычно чуть ниже тонко настроенного бустинга.

GRADIENT BOOSTING

- Часто даёт максимум качества на табличных данных.
- Требуется аккуратной совместной настройки.
- Чувствителен к шуму, может переобучаться.
- Обучается дольше, хуже параллелится.
- Нужен контроль числа деревьев и глубины.

Практический рецепт: начните с Random Forest как сильного быстрого ориентира, затем проверьте, даёт ли настроенный бустинг ощутимый прирост на честном split.



Пропуски и категориальные признаки

Практические ограничения при работе с таблицами

Деревья работают с табличными признаками естественно, но у разных реализаций – разные возможности. Важно знать практические границы, не углубляясь в детали конкретных библиотек.

ПРОПУСКИ (MISSING)

Часть реализаций обрабатывает пропуски сама (направляет объект в ветвь по обученному правилу). Другие требуют заполнить пропуски заранее. Не полагайтесь на «само разберётся» – проверьте, что делает ваша библиотека.

КАТЕГОРИАЛЬНЫЕ ПРИЗНАКИ

Классические деревья ждут числа. Категории кодируют (one-hot, ordinal); некоторые реализации (например, CatBoost) умеют работать с категориями напрямую. Кодирование делают внутри pipeline, чтобы не было утечки (как в L05).

МАСШТАБ ПРИЗНАКОВ

В отличие от линейных моделей, деревьям не нужна нормировка: разбиения по порогам инвариантны к монотонным преобразованиям. Это одно из удобств деревьев на «грязных» табличных данных.

Pipeline для деревьев и ансамблей (кодирование, обработка пропусков без утечки) детально – в L10.



Impurity-based feature importance

ОПРЕДЕЛЕНИЕ

Impurity-based feature importance

Важность признака = суммарное уменьшение impurity (Gini или энтропии) по всем разбиениям, где этот признак использовался, усреднённое по деревьям ансамбля. Чем больше признак «очищал» узлы, тем выше его важность.

Зачем это удобно. Importance считается почти бесплатно прямо при обучении и даёт быстрый ответ на вопрос «на какие признаки модель опирается сильнее всего». Это полезный ориентир: например, в ИБ он помогает заметить, что модель неожиданно сильно завязана на один признак. Но у этой простой меры есть встроенные ограничения – о них следующий слайд.



Что эта мера не показывает

Impurity-importance проста и быстра, но именно из-за простоты систематически искажает картину. Два ограничения нужно помнить всегда.

1 Смещение к признакам с многими уровнями

Признаки с большим числом значений (например, числовые с множеством порогов или категории с многими уровнями) дают больше возможностей для разбиений и потому получают завышенную важность – даже если по сути не информативнее простых.

2 Нечувствительность к корреляциям

Если два признака сильно коррелируют, дерево использует один и приписывает важность ему, а «дублёр» выглядит неважным. Это не значит, что второй признак бесполезен – просто его вклад «забрал» первый.

→ **Дальше – в L09.** Здесь importance – только концепция-иллюстрация. Надёжные методы интерпретации (permutation importance, SHAP, LIME) и анализ ошибок систематически разбираются в следующей лекции.



Частая ошибка: «важность = причина»

Importance ≠ causation

⚠ ЧАСТАЯ ОШИБКА

«Признак на первом месте по importance – значит, он причина события». Высокую важность читают как доказательство влияния признака на целевую переменную в реальном мире.

ПОЧЕМУ ЭТО НЕВЕРНО

importance показывает, на что опиралась модель, а не что влияет на мир. Это статистическая ассоциация внутри конкретной обученной модели, а не причинная связь. Признак может быть важным, потому что коррелирует с настоящей причиной, потому что в нём спрятана утечка, или просто из-за смещения impurity-меры (слайд 43). Причинность так не доказывается: для этого нужны другие методы и, как правило, эксперимент. В L08 importance – лишь сигнал «присмотреться к признаку», не вывод.



Когда «лучший» признак выдаёт ответ

ИБ-КЕЙС

Ситуация. Бустинг детектирует заражённые хосты по сетевым признакам и показывает accuracy 0.995. По importance с огромным отрывом лидирует признак `alert_count_24h` – число алертов IDS по хосту за сутки.

Разгадка. Этот признак считался уже после того, как IDS пометил хост как заражённый, – то есть частично содержит сам ответ. Это утечка по времени (как в L05). «Самый важный признак» оказался не находкой, а дырой в постановке: на новых данных, где разметка ещё не готова, признак недоступен и качество рушится.

Мораль: подозрительно доминирующий признак – повод проверить его на утечку, а не радоваться. Importance здесь сработала как **сигнал тревоги**, а не как доказательство качества (линия leakage: L05 → L09 → L19).



Выбор модели и важность признаков

RF и GB – сильные ансамбли с разным характером: лес быстр и устойчив «из коробки», бустинг точнее, но требует настройки. Сравнивайте честно, а не по репутации.

Деревьям не нужна нормировка; пропуски и категории зависят от реализации, кодирование – внутри pipeline без утечки (детали в L10).

Impurity-importance = суммарное снижение impurity по признаку. Удобна и почти бесплатна, но смещена к признакам с многими уровнями и нечувствительна к корреляциям.

Importance ≠ причинность: это опора модели, а не влияние на мир. В L08 – только концепция; методы интерпретации (permutation, SHAP, LIME) – в L09.

Подозрительно важный признак – повод проверить утечку, а не повод радоваться качеству.



Датасет CIC-IDS-2018

47 / 59

Часть 4 · Связки с практикой ИБ

Знакомая ИБ-задача, к которой мы возвращаемся

CIC-IDS-2018: ЧТО ЭТО

Публичный датасет сетевого трафика: около 16 млн flow-записей за 10 дней, 80 признаков на поток, 14 типов атак плюс класс Benign. **Это сквозной пример нашего курса:** одна и та же задача обнаружения подозрительного трафика решается всё более продвинутыми инструментами – от EDA (L02) к линейным моделям (L06), а теперь к ансамблям.

~16 млн

flow-записей

80

признаков на поток

14 + 1

типов атак и Benign



Сильное качество на табличных flow-признаках

Задача: по 80 агрегатным признакам потока (длительность, число пакетов, средний размер, флаги, межпакетные интервалы) отличить атаку от нормального трафика. Это ровно тот тип данных, где ансамбли деревьев сильны: признаки нелинейны и взаимодействуют.

ИБ-КЕЙС · ТИПИЧНЫЙ РЕЗУЛЬТАТ

Gradient Boosting на части атак (например, DDoS, brute-force) обычно достигает очень высокого качества: precision и recall в районе 0.97-0.99 на хорошо представленных классах. **Но это качество имеет смысл только при двух условиях:** разбиение сделано честно (без утечки по времени и по группам, L05) и результат сравнён с линейным baseline на тех же признаках. Без этих проверок «0.99» – не доказательство.



Всегда сравнивайте с baseline

49 / 59

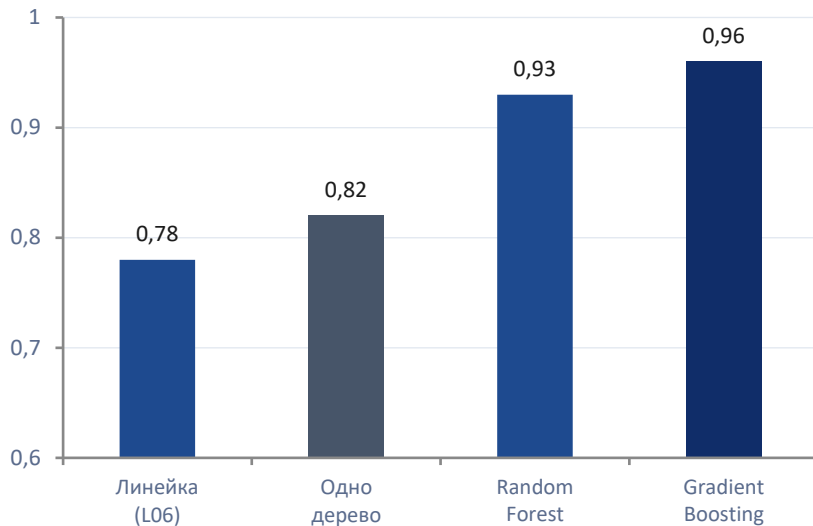
Часть 4 · Связки с практикой ИБ

Линейка из L06 – на тех же признаках, том же split

Высокое качество ансамбля само по себе ничего не говорит, пока нет точки отсчёта. **baseline-дисциплина** (линия курса L03 → L06 → L08): сначала простое дерево и линейная модель из L06, затем – лес и бустинг. Прирост считаем на одном и том же честном split.

Если бустинг обгоняет baseline на доли процента – стоит ли его сложность того? Если на 10-15 пунктов – выигрыш реален. Без baseline этот вопрос даже не поставить.

F1 на CIC-IDS-2018 (иллюстративно)



Без baseline нельзя сказать, оправдана ли сложная модель.



Где ансамбли особенно легко обманывают

Ансамбли быстро дают сильное число – и так же быстро прячут проблемы постановки. В ИБ это особенно опасно.

Утечки прячутся за высоким числом

Мощная модель легко цепляется за признак-утечку (метку времени, ID сессии, пост-фактумный счётчик). Чем выше «0.99», тем подозрительнее, если split не проверен.

Дисбаланс классов

Атаки – редкость. ассигуру обманчива: модель «всё Benign» даёт 0.99. Смотрим precision/recall по классу атаки, а не общую точность (L04).

Временная и групповая структура

Потоки одной сессии/хоста похожи. Случайный split «размазывает» их по train и test → утечка. Нужен time-based / group split (L05).



Сигнал для расследования, не вывод

В ИБ важность признаков полезна именно как **подсказка, куда смотреть**. Если модель детекции сильно опирается на неожиданный признак – это повод для аналитика разобраться, а не готовый вывод о причине атаки.

КАК ЧИТАТЬ IMPORTANCE В SOC

Полезно: заметить доминирующий признак и проверить его на утечку; увидеть, что детектор завязан на легко подделываемый признак (риск обхода). **Нельзя:** объявлять важный признак «причиной» инцидента или строить на impurity-importance защитные решения без проверки. Это смещённая и модель-зависимая мера.

→ Надёжные методы (permutation importance, SHAP, LIME, error analysis по группам алертов) – в L09. Это продолжение сквозной линии «интерпретируемость как инструмент»: L06 → L08 → L09 → L14 → L15.



Тихая утечка через подбор гиперпараметров

ЧАСТАЯ ОШИБКА

«Перебираем `learning_rate`, `depth`, число деревьев, оставляем те, что дали лучший результат на `test`». `Test` используют как площадку для настройки.

ПОЧЕМУ ЭТО НЕВЕРНО

подбор по test превращает его в часть обучения – это утечка (L05). У бустинга гиперпараметров много, и при достаточном переборе на `test` всегда найдётся «удачная» комбинация, которая просто подогналась под эту конкретную выборку. Итоговая метрика выходит оптимистичной, а в проде модель работает хуже. Правильно: настраивать на `validation` (или по кросс-валидации), а `test` трогать один раз – для финальной оценки. На ИБ-данных `split` при этом должен быть `time-based / group`, а не случайным.



Связи деревьев и ансамблей с курсом

Деревья и ансамбли – узел, от которого расходятся несколько линий курса. Что читать дальше, чтобы достроить картину:

L09

Интерпретируемость и анализ ошибок. Систематические методы: permutation importance, SHAP, LIME, error analysis по группам. Прямое продолжение слайдов про importance.

L10

Подготовка признаков и pipeline. Кодирование категорий, обработка пропусков, защита от утечки на fit_transform – техническая реализация для деревьев.

L13

Основы нейросетей. Позиция курса: MLP должен сравниваться с бустингом на табличных задачах, а не браться по умолчанию.

L17

Устойчивость под drift. Что происходит с деревьями и их importance, когда распределение данных в эксплуатации сдвигается.



7 ключевых утверждений

- 1 Дерево разбивает пространство признаков правилами «если-то»; качество разбиения мерят impurity – Gini или энтропией.
- 2 Одиночное дерево переобучается; растущий зазор train – val – главный сигнал. Контроль: глубина, размер листа, pruning.
- 3 Random Forest = bagging + случайные признаки. Усреднение декоррелированных деревьев снижает разброс, работает «из коробки».
- 4 Gradient Boosting строит деревья последовательно, исправляя ошибки; мощнее, но чувствителен к настройке.
- 5 RF и GB сравнивают честно, а не по репутации; «бустинг всегда лучше» – миф.
- 6 Feature importance – полезный, но смещённый и не причинный ориентир. Систематическая интерпретация – в L09.
- 7 Любую сложную модель сравниваем с линейным baseline на честном split. Высокая метрика без проверки – не доказательство.



Где применять, где нет

55 / 59

Часть 5 · Итоги

Границы применимости деревьев и ансамблей

ХОРОШО ПОДХОДЯТ

- Табличные данные с нелинейностями и взаимодействиями.
- Смешанные типы признаков, разные масштабы (нормировка не нужна).
- Задачи, где нужен сильный baseline быстро (Random Forest).
- Когда важна максимальная точность на табличке и есть время на настройку (Gradient Boosting).

ПЛОХО ПОДХОДЯТ / ОСТОРОЖНО

- Изображения, аудио, сырой текст – там сильнее нейросети (L13-L15).
- Гладкая экстраполяция за пределы обучающих данных (деревья дают ступеньки).
- Когда нужна простая, проверяемая причинная интерпретация – importance её не даёт.
- Без честного split и baseline – любой результат недостоверен.



Сквозные линии, проходящие через L08

L08 – узел нескольких сквозных линий курса. Эта карта помогает решить, что читать дальше для самостоятельной навигации.

Baseline-дисциплина

L03 → L06 → **L08** → L13 → L15

Простое дерево как baseline для табличных задач.

Интерпретируемость

L06 → **L08** → L09 → L14 → L15

impurity importance → систематические методы в L09.

Leakage и валидация

L05 → L10 → **L08** → L09 → L19

Важный признак как сигнал утечки.

Цена ошибок и порог

L04 → L07 → **L08** → L17 → L20

Приоритизация алертов по совокупности сигналов.



Чтобы закрепить базу по деревьям и ансамблям

Hastie, Tibshirani, Friedman – The Elements of Statistical Learning

Гл. 9 (additive models, деревья CART), гл. 15 (Random Forests), гл. 10 (Boosting). Основной разбор алгоритмов этой лекции.

Bishop – Pattern Recognition and Machine Learning

Гл. 14 (Combining Models): bagging, boosting, ансамбли с вероятностной точки зрения.

Документация scikit-learn

Разделы `sklearn.tree` и `sklearn.ensemble`: `DecisionTree`, `RandomForest`, `GradientBoosting` – параметры и примеры.

Указаны конкретные главы. Книги ПУД: Bishop и Hastie-Tibshirani-Friedman – основные источники классического ML в курсе.



Для тех, кто хочет глубже, и для практики

ГЛУБЖЕ

- Breiman (2001), «Random Forests» – оригинальная статья.
- Friedman (2001), «Greedy Function Approximation» – статья про gradient boosting.
- Chen & Guestrin (2016), «XGBoost» – для общего понимания (детали реализации – вне курса).

ПРАКТИКА

- sklearn: tutorial «Decision Trees» и «Ensemble methods» с примерами кода.
- Kaggle-ноутбуки по CIC-IDS-2018: RF и GB на сетевом трафике с честным split.
- Сравнить на своей задаче: дерево → RF → GB против линейного baseline (L06).

Граница курса: *полный разбор всех параметров CatBoost, XGBoost и LightGBM, а также продвинутый hyperparameter optimization – вне рамок L08.*



Ключевые термины

Решающее дерево – Модель из вложенных правил «если-то»; объект идёт от корня к листу.

Impurity – Мера перемешанности классов в узле; 0 – чистый узел.

Gini – $G = 1 - \sum p_k^2$; критерий разбиения.

Энтропия – $H = -\sum p_k \log_2 p_k$; критерий разбиения в битах.

Information gain – Уменьшение impurity после разбиения; критерий выбора.

Переобучение – Подгонка под шум: train высоко, validation падает.

Bagging – Обучение деревьев на bootstrap-выборках и усреднение.

Random Forest – Bagging + случайное подмножество признаков в узле.

OOB-оценка – Проверка по объектам вне bootstrap соответствующих деревьев.

Boosting – Последовательное построение ансамбля, исправляющего ошибки.

Gradient Boosting – Бустинг по антиградиенту функции потерь.

Learning rate (v) – Вес каждого нового дерева в бустинге.

Feature importance – Вклад признака; impurity-based – смещён, не причинный.

max_depth – Ограничение глубины дерева для контроля сложности.